ICPC Code Template

喵喵喵幼儿园

5 CATEGORIES	8 TEMPLATES	272 SOURCE LINES

Quick Index

1 Data Structures

1.1 01 树状数组

FILE data-structures/fwt_bitset.cpp **LINES** 41

USE 下标从 1 开始，激活位置并查询区间内已激活位置数 **COST** Add/query O(log(N/64)), in-block query O(1)

AUTHOR ppip

```
struct fwt {
    static constexpr int B{(N>>6)+5};
    int t[B],n;
    unsigned long long b[B];
    void clear() {
        n=(::n>>6)+1;
        fill_n(t,n+1,0);
        fill_n(b,n+1,0);
    }
    int px(int x) {
        int y{x&63},z{x>>6};
        return __builtin_popcountll(b[z]&(~0ull>>(63-y)));
    }
    int qry(int l,int r) {
        --l;
        int ans{px(r)-px(l)};
        l>>=6;r>>=6;
        while (r>l) ans+=t[r],r&=r-1;
        while (l>r) ans-=t[l],l&=l-1;
        return ans;
    }
    void add(int x) {
        unsigned long long w{1ull<<(x&63)};
        if (b[x>>6]&w) return;
        b[x>>6]|=w;
        x=(x>>6)+1;
        while (x<=n) {
            ++t[x];
            x+=x&-x;
        }
    }
    int qry(int x) {
        int ans{px(x)};
        x>>=6;
        while (x) {
            ans+=t[x];
            x&=x-1;
        }
        return ans;
    }
} t;
```

1.2 O(1) 查询树状数组

FILE data-structures/fwt_suffix.cpp **LINES** 16

USE 下标从 1 开始，单点加、查询后缀和；适合查询远多于修改 **COST** Query O(1), add O(128+N/4096)

AUTHOR ppip

```
struct FWT {
    int s1[N+5],s2[(N>>6)+5],s3[(N>>12)+5];
    void init() {
        memset(s1,0,sizeof(s1));
        memset(s2,0,sizeof(s2));
        memset(s3,0,sizeof(s3));
    }
    void add(int x,int y) {
        for (int i{(x>>6)<<6};i<=x;++i) s1[i]+=y;
        for (int i{(x>>12)<<6};i<(x>>6);++i) s2[i]+=y;
        for (int i{0};i<(x>>12);++i) s3[i]+=y;
    }
    int qry(int x) {
        return s1[x]+s2[x>>6]+s3[x>>12];
    }
} T;
```

1.3 Top Tree

FILE data-structures/toptree.cpp **LINES** 60

USE Magic tree that can fulfill every wish. **COST** Tree height O(log n)

AUTHOR ppip

```cpp
vector<pair<int,int>> e[N+5];
int sz[N+5],fa[N+5],son[N+5];
void TOPTREE_INIT(int u) {
    sz[u]=1;
    for (auto [v,_]:e[u])
        if (v!=fa[u]) {
            TOPTREE_INIT(v);
            sz[u]+=sz[v];
            if (sz[v]>sz[son[u]]) son[u]=v;
        }
    if (son[u]) {
        int si{exchange(sz[son[u]],0)};
        sort(e[u].begin(),e[u].end(),[](auto x,auto y){return sz[x.first]<sz[y.first];});
        sz[son[u]]=si;
    }
}
struct cluster {
    //....
} t[N*2+5];
int cnode;
using P=pair<int,int>;
using iter=vector<P>::iterator;
int merge(iter L,iter R,char tp) {
    if (L+1==R) {
        int z{L->first};
        if (z<=n) t[z].u=fa[z],t[z].v=z;
        return z;
    }
    iter p{L+1};
    int sum{accumulate(p,R,-L->second,[](int x,P y){return x+y.second;})};
    while (p+1!=R&&sum>p->second) {
        sum-=p->second*2;
        ++p;
    }
    int u{++cnode};
    t[u].t=tp;
    t[u].l=merge(L,p,tp);
    t[u].r=merge(p,R,tp);
    t[t[u].l].f=t[t[u].r].f=u;
    t[u].u=t[t[u].l].u;
    t[u].v=(tp=='C'?t[t[u].r].v:t[t[u].l].v);
    return u;
}
int build(int u) {
    vector<P> v1;v1.emplace_back(u,1);
    while (son[u]) {
        vector<P> v2;v2.emplace_back(son[u],1);
        for (auto [v,_]:e[u])
            if (v!=fa[u]&&v!=son[u]) v2.emplace_back(build(v),sz[v]);
        v1.emplace_back(merge(v2.begin(),v2.end(),'R'),sz[u]-sz[son[u]]);
        u=son[u];
    }
    return merge(v1.begin(),v1.end(),'C');
}
int main() {
    cnode=n;
    TOPTREE_INIT(1);
    int rt{build(1)};
    return 0;
}
```

2 Graph / Flow

2.1 最大流

FILE graph/flow/maxflow.cpp **LINES** 60

USE Dinic, 要求汇点编号为最大值 **COST** 相信的心就是魔法

AUTHOR ppip

```cpp
struct edge {
    int u,v,w,nxt;
    edge(int _,int __,int ___,int ____):
        u{_},v{__},w{___},nxt{____} {}
    edge() {}
};
vector<edge> e(2);
int hd[N+5];
void _add(int u,int v,int w) {
    e.emplace_back(u,v,w,hd[u]);
    hd[u]=e.size()-1;
}
void add(int u,int v,int w) {
    _add(u,v,w);
    _add(v,u,0);
}
int q[N+5];
int dep[N+5],cur[N+5];
bool bfs(int s,int t) {
    dep[s]=1;
    int l{0},r{0};
    q[r++]=s;
    while (r-l) {
        int u{q[l++]};
        for (int x{hd[u]};x;x=e[x].nxt)
            if (!dep[e[x].v]&&e[x].w) {
                dep[e[x].v]=dep[u]+1;
                q[r++]=e[x].v;
            }
    }
    return dep[t];
}
long long dfs(int s,int t,long long flow) {
    if (s==t||!flow) return flow;
    long long ans{0};
    for (int &x{cur[s]},d;x;x=e[x].nxt) {
        if (dep[e[x].v]==dep[s]+1&&e[x].w&&(d=dfs(e[x].v,t,min(flow,(long long)e[x].w)))) {
            flow-=d;
            ans+=d;
            e[x].w-=d;
            e[x^1].w+=d;
            if (!flow) return ans;
        }
    }
    return ans;
}
long long calc(int s,int t) {
    fill_n(dep+1,t,0);
    long long flow{0};
    while (bfs(s,t)) {
        copy(hd+1,hd+t+1,cur+1);
        flow+=dfs(s,t,LLONG_MAX);
        fill_n(dep+1,t,0);
    }
    return flow;
}
void clear(int t) {
    e.resize(2);
    fill_n(hd+1,t,0);
}
```

3 Math

3.1 Binary GCD

FILE math/binary_gcd.cpp **LINES** 6

USE 超快 gcd 算法 **COST** O(log n)，可以当作 O(1)

AUTHOR ppip

```cpp
#define ctz __builtin_ctz
inline int gcd(int a,int b) {
    int az{ctz(a)},bz{ctz(b)},z{min(az,bz)},t;b>>=bz;
    while (a) a>>=az,t=a-b,az=ctz(t),b=min(a,b),a=abs(t);
    return b<<z;
}
```

4 Misc

4.1 快速模乘

FILE misc/fastmod.cpp **LINES** 16

USE 要求 1<=P<2^31 且 a<P；同一 z 多次使用时先 trans，再用 mul_t **COST** O(1)

AUTHOR ppip

```cpp
using u32=unsigned;
using u64=unsigned long long;
using u128=unsigned __int128;
const u32 P=998244353;
inline u64 trans(u64 x) {
    constexpr u64 A=-(u64)P/P+1;
    constexpr u64 q=((u128(-(u64)P%P)<<64)+P-1)/P;
    return x*A+u64((u128)x*q>>64)+1;
}
// t 必须是 trans(z) 的返回值；复用同一 z 时只需计算一次 t
inline u32 mul_t(u32 a,u64 t) {
    return a*t*(u128)P>>64;
}
inline u32 mul(u32 a,u64 z) {
    return mul_t(a,trans(z));
}
```

5 String

5.1 后缀数组

FILE string/sa.cpp **LINES** 34

USE 倍增算法求 sa，rk，height, 字符必须不超过 '~' **COST** O(n log n)

AUTHOR ppip

```cpp
char s[N+5];
int id[N*2+5],sa[N+5],buc[N+5],h[N+5],rk[N+5];
int main() {
    // read s
    int n(strlen(s+1));
    for (int i{1};i<=n;++i) ++buc[s[i]];
    partial_sum(buc+1,buc+1+'~',buc+1);
    for (int i{n};i>=1;--i) sa[buc[s[i]]--]=i;
    int m{0};
    for (int i{1};i<=n;++i) rk[sa[i]]=m+=(s[sa[i]]!=s[sa[i-1]]);
    fill_n(buc+1,'~',0);
    for (int w{1};m<n;w<<=1) {
        int cnt{0};
        for (int i{0};i<w;++i) id[++cnt]=n-i;
        for (int i{1};i<=n;++i) {
            if (sa[i]>w) id[++cnt]=sa[i]-w;
            ++buc[rk[i]];
        }
        partial_sum(buc+1,buc+1+m,buc+1);
        for (int i{n};i>=1;--i) sa[buc[rk[id[i]]]--]=id[i];
        fill_n(buc+1,m,0);
        copy_n(rk+1,n,id+1);
        m=0;
        for (int i{1};i<=n;++i)
            rk[sa[i]]=m+=(id[sa[i]]!=id[sa[i-1]]||id[sa[i]+w]!=id[sa[i-1]+w]);
    }
    // for (int i{1};i<=n;++i) cout<<sa[i]<<" ";
    for (int i{1},k{0};i<=n;++i) {
        k-=!!k;
        while (s[i+k]==s[sa[rk[i]-1]+k]) ++k;
        h[rk[i]]=k;
    }
    return 0;
}
```

5.2 后缀树

FILE string/sft.cpp **LINES** 39

USE Ukkonen 在线构造隐式后缀树 **COST** build O(n)

AUTHOR ppip

```cpp
constexpr int N(1e6),inf(1e7),RNG{26+26+10};
struct Suft {
```

```
    int ch[N*2+5][RNG];
    int st[N*2+5],len[N*2+5],link[N*2+5];
    char s[N+5];
    int now{1},rem{0},n{0},tot{1};
    Suft() {len[0]=inf;}
    int new_node(int l,int le) {
        ++tot;st[tot]=l;len[tot]=le;
        return tot;
    }
    void extend(int x) {
        s[++n]=x;++rem;
        for (int lst{1};rem;) {
            while (rem>len[ch[now][s[n-rem+1]]])
                rem-=len[now=ch[now][s[n-rem+1]]];
            int &v{ch[now][s[n-rem+1]]},c{s[st[v]+rem-1]};
            if (!v||x==c) {
                link[lst]=now;lst=now;
                if (!v) v=new_node(n,inf);
                else break;
            } else {
                int u{new_node(st[v],rem-1)};
                ch[u][c]=v;ch[u][x]=new_node(n,inf);
                st[v]+=rem-1;len[v]-=rem-1;
                link[lst]=v=u;lst=u;
            }
            if (now==1) --rem;
            else now=link[now];
        }
    }
    void search(int u,int dep=0) {
        if (st[u]+len[u]-1>=n&&st[u]-dep!=n) cout<<st[u]-dep<<" ";
        else dep+=len[u];
        for (int i{0};i<RNG;++i)
            if (ch[u][i]) search(ch[u][i],dep);
    }

} T;
```

警钟长鸣

读题与建模

! 多模拟样例

思考误区

! 发现新性质后，回头检查旧做法能否简化或是否已经不成立

! DP 先想清楚有哪些可靠的状态和存储方式，再设计转移

实现与边界

! 任意范围修改、查询时，考虑复杂度会不会退化

提交之前

! 检查多测清空

! 检查 long long 和取模

! TLE 时重新检查实际复杂度，尤其注意分治树等结构是否退化